bioeng-qpcr

使用说明与能力清单

v0.5.0 | 2026 年 8 月

877	测试通过
88%	代码覆盖率
15	MCP ■■
4	■■■■

文档说明

bioeng-qpcr 是面向生物工程研究的可审计分析引擎，覆盖 qPCR 数据验证、酶动力学、发酵动力学和 FBA 代谢通量分析。本手册详细介绍各模块的功能、数据格式和使用方法。

目录

1 项目概览与架构

2 安装与环境配置

3 输入数据格式 (Level A CSV)

4 模块一：qPCR 分析

5 模块二：酶动力学

6 模块三：发酵动力学

7 模块四：FBA 代谢通量

8 MIQE 验证规则清单

9 MCP Server 工具清单

10 CLI 命令速查

1 项目概览与架构

bioeng-qpcr 是一个面向生物工程研究的可审计分析引擎，覆盖从 qPCR 数据验证到代谢通量分析的完整链路。所有分析结果均携带来源追踪信息（provenance），确保可复现性。

核心设计原则

- 校验与计算分离：校验层（MIQE 规则）独立于计算层，可单独调用
- 核心库独立：纯 Python 核心库不依赖 GUI 或 AI 框架，可独立发布
- 三层入口：CLI（命令行）、Python API（编程）、MCP Server（AI 助手）三种使用方式
- 可审计性：每次分析自动生成 provenance manifest（输入哈希 + 包版本 + 参数 + 时间戳）

模块架构

模块	包路径	功能	可选依赖
qPCR 核心	bioeng_qpcr.calculations	ΔCq / ΔΔCq / Pfaffl / 效率校正归一化	—
MIQE 验证	bioeng_qpcr.validation	7 条规则的自动化合规检查	—
参考基因	bioeng_qpcr.reference_genes	geNorm / NormFinder / 共识排名	—
统计	bioeng_qpcr.statistics	置信区间 / t 检验 / Bootstrap	—
可视化	bioeng_qpcr.plotting	柱状图 / 热图 / 生长曲线 / 通量图	matplotlib
酶动力学	bioeng_qpcr.enzyme_kinetics	Michaelis-Menten 拟合 / 抑制类型判别	—
发酵动力学	bioeng_qpcr.fermentation_kinetics	Monod ODE / 参数可辨识性	—
FBA 代谢通量	bioeng_qpcr.metabolic_flux	FBA 求解 / 阻塞反应 / 死端代谢物	cobra
Provenance	bioeng_qpcr.provenance	来源追踪 manifest	—
报告	bioeng_qpcr.reporting	HTML 研究报告生成	matplotlib
MCP Server	bioeng_qpcr.mcp	15 个 AI 工具	mcp, pydantic

2 安装与环境配置

要求

- Python ≥ 3.10
- 核心依赖：numpy, scipy, pandas, click
- 可选依赖：matplotlib（绘图）, cobra（FBA）, mcp + pydantic（AI 工具）

安装方式

```
#
pip install -e ".[dev]"

#
pip install -e "."

#
pip install -e ".[plot]"      #
pip install -e ".[mcp]"       # MCP Server
pip install -e ".[fba]"       # FBA
pip install -e ".[widgets]"   # ipywidgets
```

验证安装

```
python -m bioeng_qpcr.cli.main --version
#   : bioeng-qpcr 0.5.0
```

> 注意
> 如果终端中 python 指向 Python 2.x，需要使用 Python 3 的完整路径，或调整 PATH 优先级。

3 输入数据格式 (Level A CSV)

qPCR 模块使用 Level A Tidy CSV 格式。每行代表一个孔（well），必须包含以下列：

```
position,sample_name,target_name,cq,replicate_type,replicate_group
A01,S1,GAPDH,20.5,technical,S1_GAPDH
A02,S1,GAPDH,20.7,technical,S1_GAPDH
A03,S1,GAPDH,20.6,technical,S1_GAPDH
A04,S1,ACTB,18.3,technical,S1_ACTB
A05,S1,ACTB,18.5,technical,S1_ACTB
A06,S1,ACTB,18.4,technical,S1_ACTB
A07,S2,GAPDH,21.0,technical,S2_GAPDH
...
```

列说明

列名	必填	说明
position	是	孔位编号（如 A01）
sample_name	是	样本名称
target_name	是	检测目标基因名称
cq	是	Cq/Ct 值（数值或 Undetermined）
replicate_type	否	重复类型：technical 或 biological
replicate_group	否	重复组标识符

可选的 Assay 元数据列

在 CSV 中添加以下列可补充实验元数据（用于 MIQE 验证）：

- efficiency — 扩增效率（如 0.95 表示 95%）
- r2 — 标准曲线 R^2
- slope — 标准曲线斜率
- is_reference — 是否为参考基因（True/False）
- lod_cq — 检测限 Cq
- lloq_cq — 定量下限 Cq

4 模块一：qPCR 分析

支持 CLI、Python API、MCP 三种使用方式。

4.1 MIQE 就绪评估

对 tidy CSV 文件运行 7 条 MIQE 合规规则检查，输出 PASS / WARN / FAIL / NOT_ASSESSABLE 状态。

CLI

```
python -m bioeng_qpcr.cli.main validate data.csv
python -m bioeng_qpcr.cli.main validate data.csv --reference-targets GAPDH,RPLP0
python -m bioeng_qpcr.cli.main validate data.csv --lenient
```

Python API

```
from bioeng_qpcr.io import read_tidy_csv
from bioeng_qpcr.validation import ValidationEngine

experiment = read_tidy_csv("data.csv", reference_targets=["GAPDH"])
engine = ValidationEngine()
report = engine.run(experiment)

for result in report.results:
    print(f"{result.rule_id}: {result.status} - {result.message}")
```

4.2 相对定量分析

支持三种定量方法：

方法	说明	需要效率	需要对照
normalized_quantity	效率校正归一化（推荐）	是	否
pfaffl	Pfaffl 效率校正法	是	是
delta_delta_cq	ΔΔCq (Livak) 法，假设 100% 效率	否	是

CLI

```
#
python -m bioeng_qpcr.cli.main analyze data.csv \
    --target IL6 --method normalized_quantity \
    --reference-targets GAPDH,RPLP0

# ΔΔCq
python -m bioeng_qpcr.cli.main analyze data.csv \
    --target ACTB --method delta_delta_cq \
    --control S1 --reference-targets GAPDH

#     CSV
python -m bioeng_qpcr.cli.main analyze data.csv \
    --target IL6 --output csv --output-file results.csv
```

Python API

```
from bioeng_qpcr.io import read_tidy_csv
from bioeng_qpcr.calculations import normalized_quantity_all, delta_delta_cq_batch

experiment = read_tidy_csv("data.csv", reference_targets=["GAPDH"])

#
results = normalized_quantity_all(experiment, target="IL6")
for r in results:
    print(f"{r.sample_name}: NQ={r.normalized_quantity:.4f}")

# ΔΔCq
results = delta_delta_cq_batch(experiment, target="ACTB", control="S1")
for r in results:
    print(f"{r.sample_name}: fold_change={r.fold_change:.4f}")
```

4.3 分组比较

```
# CLI
python -m bioeng_qpcr.cli.main compare data.csv \
    --target IL6 --treated-group treated --control-group control \
    --reference-targets GAPDH

# Python API
from bioeng_qpcr.calculations import compare_groups
result = compare_groups(experiment, target="IL6",
                        treated_group="treated", control_group="control")
print(f"Fold difference: {result.fold_difference:.4f}")
```

4.4 参考基因稳定性

```
from bioeng_qpcr.reference_genes import genorm, normfinder, consensus_ranking

# geNorm
geo_result = genorm(experiment, reference_targets=["GAPDH", "RPLP0"])

# NormFinder
nf_result = normfinder(experiment, reference_targets=["GAPDH", "RPLP0"])

#
consensus = consensus_ranking(experiment, reference_targets=["GAPDH", "RPLP0"])
```

4.5 可视化

```
# CLI
python -m bioeng_qpcr.cli.main plot data.csv --type fold_change \
    --target ACTB --control S1 --reference-target GAPDH \
    --output-file fold_change.png
python -m bioeng_qpcr.cli.main plot data.csv --type heatmap \
    --output-file heatmap.png

# Python API
from bioeng_qpcr.plotting import plot_fold_change, plot_cq_heatmap
fig, ax = plot_fold_change(experiment, target="ACTB", control="S1")
fig.savefig("fold_change.png")
```

4.6 HTML 研究报告

```
python -m bioeng_qpcr.cli.main report data.csv \
    --target IL6 --reference-targets GAPDH,RPLP0 \
    --output-file report.html
```

4.7 Provenance 来源追踪

```
python -m bioeng_qpcr.cli.main provenance data.csv \
    --parameters method=normalized_quantity target=IL6 \
    --output-file provenance.json
```

5 模块二：酶动力学

支持 Python API 和 MCP 两种使用方式。

5.1 Michaelis-Menten 参数拟合

从底物浓度-反应速度数据拟合 Vmax 和 Km。

输入 CSV 格式

```
substrate_concentration,velocity
1.0,9.09
2.0,16.67
5.0,33.33
10.0,50.00
20.0,66.67
50.0,83.33
100.0,90.91
```

Python API

```
from bioeng_qpcr.enzyme_kinetics import fit_michaelis_menten, read_kinetics_csv

substrate, velocity, _ = read_kinetics_csv("enzyme_data.csv")
result = fit_michaelis_menten(substrate, velocity)

print(f"Vmax = {result.vmax:.4f}")
print(f"Km   = {result.km:.4f}")
print(f"R²   = {result.r_squared:.4f}")
print(f"Method = {result.method}")  # "leastsq"   "lineweaver_burk"
```

5.2 抑制类型判别

从不同抑制剂浓度下的动力学数据判别抑制类型：竞争性、非竞争性、反竞争性、混合型。

输入 CSV 格式

```
substrate_concentration,velocity,inhibitor_concentration
1.0,9.09,0.0
2.0,16.67,0.0
...
1.0,4.76,5.0
2.0,9.09,5.0
...
```

Python API

```
from bioeng_qpcr.enzyme_kinetics import classify_inhibition, read_kinetics_csv

substrate, velocity, inhibitor = read_kinetics_csv("inhibition_data.csv")
result = classify_inhibition(substrate, velocity, inhibitor)

print(f"Type: {result.inhibition_type}")
    # competitive / noncompetitive / uncompetitive / mixed
print(f"Ki    = {result.ki:.4f}")
print(f"Alpha Ki = {result.alpha_ki:.4f}")
```

5.3 校验规则

```
from bioeng_qpcr.enzyme_kinetics import validate_km_range, validate_substrate_span

km_check = validate_km_range(km=10.0, substrate_range=(1.0, 100.0))
span_check = validate_substrate_span(substrate=[1.0, 2.0, 5.0, 10.0, 20.0, 50.0, 100.0])
```

5.4 可视化

```
from bioeng_qpcr.plotting import plot_michaelis_menten, plot_lineweaver_burk

fig1, _ = plot_michaelis_menten(result)
fig1.savefig("mm_plot.png")

fig2, _ = plot_lineweaver_burk(substrate, velocity)
fig2.savefig("lb_plot.png")
```

6 模块三：发酵动力学

支持 Python API 和 MCP 两种使用方式。

6.1 Monod 参数拟合

从分批发酵的时间序列数据拟合 Monod 动力学参数：μmax（最大比生长速率）、Ks（半饱和常数）、Yxs（表观得率系数）。

输入 CSV 格式

```
time,biomass,substrate
0.0,0.1507,10.0484
2.2,0.2336,9.3219
4.4,0.6750,8.6371
6.5,1.6197,7.1589
8.7,3.2364,3.4770
10.9,4.9799,0.2553
...
```

Python API

```
from bioeng_qpcr.fermentation_kinetics import fit_monod, read_fermentation_csv

time, biomass, substrate = read_fermentation_csv("fermentation.csv")
result = fit_monod(time, biomass, substrate)

print(f"μmax = {result.mu_max:.6f} h⁻¹")
print(f"Ks   = {result.ks:.4f} g/L")
print(f"Yxs  = {result.yxs:.4f}")
print(f"R²   = {result.r_squared:.4f}")
```

6.2 参数可辨识性分析

通过 Fisher 信息矩阵评估参数的可辨识性。注意：单批次数据中 μmax 和 Ks 存在结构性高相关，需多批次数据才能完全辨识。

```
from bioeng_qpcr.fermentation_kinetics import analyze_identifiability

id_result = analyze_identifiability(time, biomass, substrate, result)
print(f"Condition number: {id_result.condition_number:.2f}")
print(f"Correlation μmax-Ks: {id_result.correlation_matrix[0][1]:.4f}")
```

6.3 ODE 求解与可视化

```
from bioeng_qpcr.fermentation_kinetics import solve_batch_fermentation
from bioeng_qpcr.plotting import plot_fermentation_kinetics

#         ODE
t_sol, biomass_sol, sub_sol = solve_batch_fermentation(
    mu_max=result.mu_max, ks=result.ks, yxs=result.yxs,
    s0=10.0, x0=0.15, t_end=24.0
)

#     y
fig, _ = plot_fermentation_kinetics(result, time, biomass, substrate)
fig.savefig("growth_curve.png")
```

6.4 校验规则

```
from bioeng_qpcr.fermentation_kinetics import validate_mu_max_range, validate_data_sufficiency

mu_check = validate_mu_max_range(result.mu_max)  # FERM-001:      0 < μmax < 2
data_check = validate_data_sufficiency(time, biomass, substrate)  # FERM-002:
```

7 模块四：FBA 代谢通量

> 可选依赖
> FBA 模块需要安装 COBRApy：pip install cobra。未安装时模块自动跳过，不影响其他功能。

7.1 FBA 求解

加载代谢模型（SBML/JSON/MAT 格式或内置模型如 textbook），求解 Flux Balance Analysis。

```
from bioeng_qpcr.metabolic_flux import load_model, run_fba

#      E. coli core
model = load_model("textbook")

#      SBML
model = load_model("my_model.xml")

#    FBA
result = run_fba(model)

print(f"Objective value: {result.objective_value:.6f}")
print(f"Status: {result.status}")
print(f"Active reactions: {result.n_active}/{result.n_total}")

#
result = run_fba(model, objective="ATPM", direction="max")
```

7.2 网络分析

```
from bioeng_qpcr.metabolic_flux import find_blocked_reactions, find_dead_end_metabolites

#        FVA
blocked = find_blocked_reactions(model)
print(f"Blocked: {blocked.n_blocked}/{blocked.n_total} ({blocked.fraction_blocked:.1%})")

#
dead_ends = find_dead_end_metabolites(model)
print(f"Dead-ends: {dead_ends.n_dead_ends}/{dead_ends.n_total}")
for met_id, detail in dead_ends.details.items():
    print(f"  {met_id}: {detail['type']}")  # "source" / "sink" / "isolated"
```

7.3 验证规则

```
from bioeng_qpcr.metabolic_flux import validate_objective_feasibility, validate_mass_balance

# FBA-001:
check1 = validate_objective_feasibility(result)
print(f"FBA-001: {'PASS' if check1.passed else 'FAIL'} - {check1.message}")

# FBA-002:
check2 = validate_mass_balance(model, result)
print(f"FBA-002: {'PASS' if check2.passed else 'FAIL'} - {check2.message}")
print(f"  Checked: {check2.evidence['n_metabolites_checked']} metabolites")
print(f"  Max residual: {check2.evidence['max_residual']:.2e}")
```

7.4 可视化

```
from bioeng_qpcr.plotting import plot_flux_distribution

fig, _ = plot_flux_distribution(result, top_n=20)
fig.savefig("flux_distribution.png")
```

8 MIQE 验证规则清单

内置 7 条自动化验证规则，覆盖 MIQE 2.0 核心要求：

规则 ID	名称	检查内容	状态
INPUT-001	Input Data Integrity	数据导入阶段的强制转换和排除记录	PASS / WARN
ASSAY-001	Assay Characterization	标准曲线证据覆盖度（efficiency, R^2, slope, LOD, LOQ）	PASS / WARN / NOT_ASSESSABLE
EFF-001	Amplification Efficiency	扩增效率是否在 90%-110% 范围内	PASS / FAIL / NOT_ASSESSABLE
NTC-001	No-Template Control	NTC 孔是否存在且无扩增	PASS / FAIL / NOT_ASSESSABLE
REF-001	Reference Gene Presence	是否指定了至少一个参考基因	PASS / FAIL
REF-002	Reference Gene Multiplicity	是否使用多个参考基因并验证稳定性	PASS / WARN / NOT_ASSESSABLE
REP-001	Technical Replicate Agreement	技术重复 Cq 值一致性（默认 ΔCq ≤ 0.5）	PASS / WARN

扩展规则

规则 ID	模块	检查内容
FERM-001	发酵动力学	μmax 物理约束（0 < μmax < 2 h□¹）
FERM-002	发酵动力学	数据点充分性（最少 8 个时间点）
FBA-001	FBA 代谢通量	目标可行性（status=optimal 且目标值 > 0）
FBA-002	FBA 代谢通量	质量平衡（内部代谢物稳态残差 < 1e-6）

9 MCP Server 工具清单

MCP Server 暴露 15 个工具供 AI 助手调用（核心 14 个 + FBA 1 个，FBA 需 cobra）。

工具名	功能	模块
qpcr_validate	MIQE 就绪评估	qPCR
qpcr_analyze	相对定量分析	qPCR
qpcr_compare_groups	分组比较	qPCR
qpcr_generate_report	HTML 研究报告	qPCR
qpcr_get_provenance	来源追踪 manifest	qPCR
qpcr_reference_stability	参考基因稳定性（geNorm/NormFinder/共识）	qPCR
qpcr_fit_efficiency	标准曲线效率拟合	qPCR
qpcr_plot_fold_change	fold-change 柱状图（返回 base64 PNG）	qPCR
qpcr_plot_heatmap	Cq 热图（返回 base64 PNG）	qPCR
enzyme_fit_kinetics	Michaelis-Menten 参数拟合	酶动力学
enzyme_classify_inhibition	抑制类型判别	酶动力学
enzyme_plot_kinetics	酶动力学图（返回 base64 PNG）	酶动力学
ferment_fit_monod	Monod 参数拟合	发酵动力学
ferment_plot_kinetics	发酵动力学图（返回 base64 PNG）	发酵动力学
fba_analyze	FBA 求解 + 网络分析 + 可选通量图	FBA

启动 MCP Server

```
python -m bioeng_qpcr.mcp
```

> **设计约束**
> 工具总数严格控制 ≤ 15 个。FBA 工具合并了求解、分析和可选绘图为一个工具，通过 include_plot 参数控制。

10 CLI 命令速查

命令	功能	必需参数
validate	MIQE 就绪评估	CSV_PATH
analyze	相对定量分析	CSV_PATH --target
compare	分组比较	CSV_PATH --target --treated-group --control-group
plot	绘制图表	CSV_PATH --type
report	生成 HTML 报告	CSV_PATH --target
provenance	生成 provenance JSON	CSV_PATH

通用选项

- --lenient — 宽松模式，可恢复的输入问题保留为诊断信息而非拒绝
- --allow-exclusions — 返回部分结果并报告排除的样本
- --output-file — 输出到文件而非标准输出
- --reference-targets — 指定参考基因（逗号分隔）